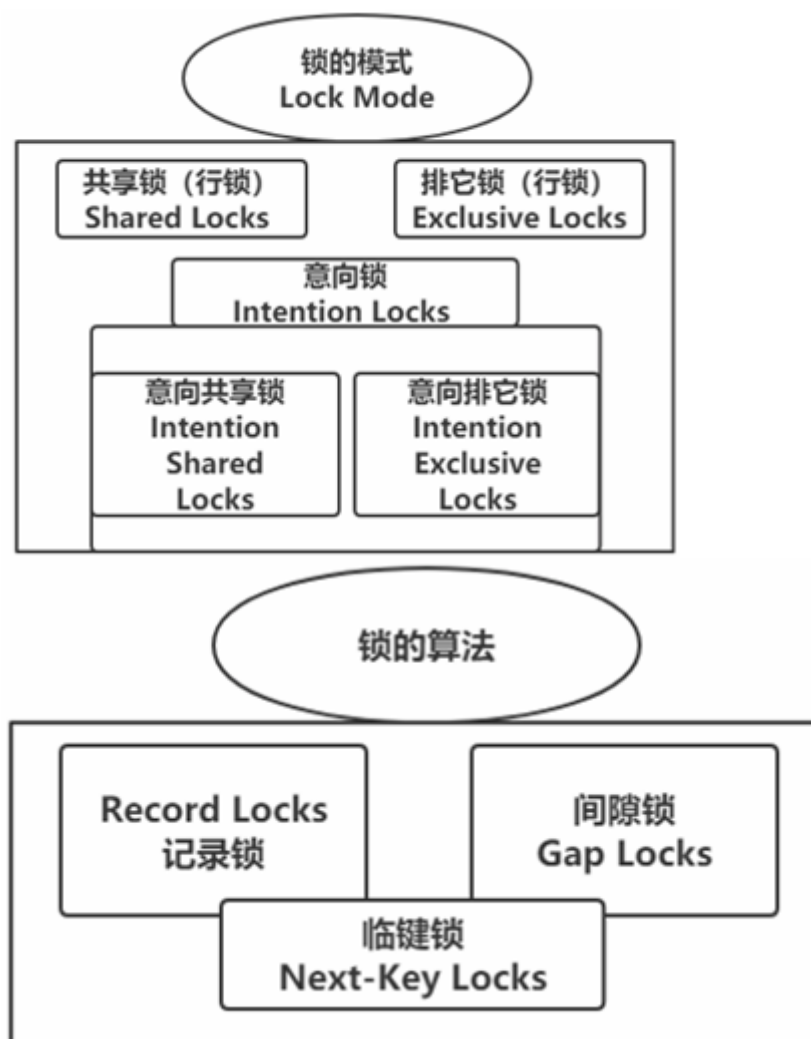


1.6. MySQL中的锁

InnoDB中锁非常多，总的来说，可以如下分类：



这些锁都是做什么的？具体含义是什么？我们现在来——学习。

1.6.1. 解决并发事务问题

我们已经知道事务并发执行时可能带来的各种问题，最大的一个难点是：一方面要最大程度地利用数据库的并发访问，另外一方面还要确保每个用户能以一致的方式读取和修改数据，尤其是一个事务进行读取操作，另一个同时进行改动操作的情况下。

1.6.2. 并发事务问题

一个事务进行读取操作，另一个进行改动操作，我们前边说过，这种情况下可能发生脏读、不可重复读、幻读的问题。

怎么解决脏读、不可重复读、幻读这些问题呢？其实有两种可选的解决方案：

1.6.2.1.方案一：读操作MVCC，写操作进行加锁

事务利用MVCC进行的读取操作称之为**一致性读**，或者**一致性无锁读**，也称之为**快照读**，但是往往读取的是历史版本数据。所有普通的SELECT语句（plain SELECT）在READ COMMITTED、REPEATABLE READ隔离级别下都算是一致性读。

一致性读并不会对表中的任何记录做加锁操作，其他事务可以自由的对表中的记录做改动。

很明显，采用MVCC方式的话，**读-写操作彼此并不冲突，性能更高**，采用加锁方式的话，**读-写操作彼此需要排队执行，影响性能**。一般情况下我们当然愿意采用MVCC来解决读-写操作并发执行的问题，但是业务在某些情况下，要求必须采用加锁的方式执行。

1.6.2.2方案二：读、写操作都采用加锁的方式

适用场景：

业务场景不允许读取记录的旧版本，而是每次都必须去读取记录的最新版本，

比方在银行存款的事务中，你需要先把账户的余额读出来，然后将其加上本次存款的数额，最后再写到数据库中。在将账户余额读取出来后，就不想让别的事务再访问该余额，直到本次存款事务执行完成，其他事务才可以访问账户的余额。这样在读取记录的时候也就需要对其进行加锁操作，这样也就意味着读操作和写操作也像写-写操作那样排队执行。

脏读的产生是因为当前事务读取了另一个未提交事务写的一条记录，如果另一个事务在写记录的时候就给这条记录加锁，那么当前事务就无法继续读取该记录了，所以也就不会有脏读问题的产生了。

不可重复读的产生是因为当前事务先读取一条记录，另外一个事务对该记录做了改动之后并提交之后，当前事务再次读取时会获得不同的值，如果在当前事务读取记录时就给该记录加锁，那么另一个事务就无法修改该记录，自然也不会发生不可重复读了。

幻读问题的产生是因为当前事务读取了一个范围的记录，然后另外的事务向该范围内插入了新记录，当前事务再次读取该范围的记录时发现了新插入的新记录，我们把新插入的那些记录称之为幻影记录。采用加锁的方式解决幻读问题就有不太容易了，因为当前事务在第一次读取记录时那些幻影记录并不存在，所以读取的时候加锁就有点麻烦——因为并不知道给谁加锁。InnoDB中是如何解决的，我们后面会讲到。

1.6.3锁定读（LockingReads）/LBCC

也称当前读，读取的是最新版本，并且对读取的记录加锁，阻塞其他事务同时改动相同记录，避免出现安全问题。

哪些是当前读呢？

- select lock in share mode (共享锁)、
- select for update (排他锁)、
- update (排他锁)、
- insert (排他锁/独占锁)、
- delete (排他锁)、
- 串行化事务隔离级别都是当前读。

当前读这种实现方式，也可以称之为LBCC（基于锁的并发控制，Lock-Based Concurrency Control），怎么做到？

1.6.3.1. 共享锁和独占锁

在使用加锁的方式解决问题时，由于既要允许读-读情况不受影响，又要使写-写、读-写或写-读情况中的操作相互阻塞，MySQL中的锁有好几类：

共享锁英文名：Shared Locks，简称S锁。在事务要读取一条记录时，需要先获取该记录的S锁。

假如事务E1首先获取了一条记录的S锁之后，事务E2接着也要访问这条记录：

如果事务E2想要再获取一个记录的S锁，那么事务E2也会获得该锁，也就意味着事务E1和E2在该记录上同时持有S锁。

独占锁，也常称**排他锁**，英文名：Exclusive Locks，简称X锁。在事务要改动一条记录时，需要先获取该记录的X锁。

如果事务E2想要再获取一个记录的X锁，那么此操作会被阻塞，直到事务E1提交之后将S锁释放掉。

如果事务E1首先获取了一条记录的X锁之后，那么不管事务E2接着想获取该记录的S锁还是X锁都会被阻塞，直到事务E1提交。

所以我们说S锁和S锁是兼容的，S锁和X锁是不兼容的，X锁和X锁也是不兼容的，画个表表示一下就是这样：

锁	是否兼容X	是否兼容S
X	不兼容X	不兼容S
S	不兼容X	兼容S

1.6.3.2. 锁定读的SELECT语句

MySQL有两种比较特殊的SELECT语句格式：

```
SELECT * from test LOCK IN SHARE MODE;
```

一个事务中开启S锁

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from teacher where number = 30 LOCK IN SHARE MODE;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      30 | 豹    | RocketMQ |
+-----+-----+-----+
1 row in set (0.00 sec)
```

另一个事务中开启S锁，可以读

```
mysql> select * from teacher where number = 30 LOCK IN SHARE MODE;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      30 | 豹    | RocketMQ |
+-----+-----+-----+
1 row in set (0.00 sec)
```

如果另外一个事务中开启X锁，阻塞！

```
mysql> select * from teacher where number = 30 for update;
```

也就是在普通的SELECT语句后边加LOCK IN SHARE MODE，如果当前事务执行了该语句，那么它会为读取到的记录加S锁，这样允许别的事务继续获取这些记录的S锁（比方说别的事务也使用SELECT ... LOCK IN SHARE MODE语句来读取这些记录），但是不能获取这些记录的X锁（比方说使用SELECT ... FOR UPDATE语句来读取这些记录，或者直接修改这些记录）。

如果别的事务想要获取这些记录的X锁，那么它们会阻塞，直到当前事务提交之后将这些记录上的S锁释放掉。

对读取的记录加X锁：

```
SELECT * from test FOR UPDATE;
```

也就是在普通的SELECT语句后边加FOR UPDATE，如果当前事务执行了该语句，那么它会为读取到的记录加X锁，这样既不允许别的事务获取这些记录的S锁（比方说别的事务使用SELECT ... LOCK IN SHARE MODE语句来读取这些记录），也不允许获取这些记录的X锁（比如说使用SELECT ... FOR UPDATE语句来读取这些记录，或者直接修改这些记录）。

一个事务中开启X锁

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from teacher where number = 30 for update;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      30 | 豹    | RocketMQ |
+-----+-----+-----+
1 row in set (0.00 sec)
```

另外一个事务中的X锁阻塞

```
mysql>
mysql> select * from teacher where number = 30 for update;
```

除非第一个事务提交

```
mysql> commit;
Query OK, 0 rows affected (0.00 sec)
```

另外一个事务才能获得X锁

```
mysql>
mysql> select * from teacher where number = 30 for update;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
|      30 | 豹    | RocketMQ |
+-----+-----+-----+
1 row in set (47.77 sec)
```

同样如果另外一个事务执行X锁，使用S锁也不行

```
mysql> select * from teacher where number = 30 LOCK IN SHARE MODE;
```

如果别的事务想要获取这些记录的S锁或者X锁，那么它们会阻塞，直到当前事务提交之后将这些记录上的X锁释放掉。

1.2.1.3. 写操作的锁（x锁）

平常所用到的写操作无非是DELETE、UPDATE、INSERT这三种：

DELETE：

对一条记录做DELETE操作的过程其实是先在B+树中定位到这条记录的位置，然后获取一下这条记录的X锁，然后再执行delete mark操作。我们也可以把这个定位待删除记录在B+树中位置的过程看成是一个获取X锁的锁定读。

INSERT：

一般情况下，新插入一条记录的操作并不加锁，InnoDB通过一种称之为隐式锁来保护这条新插入的记录在本事务提交前不被别的事务访问。当然，在一些特殊情况下INSERT操作也是会获取锁的，具体情况我们后边再说。

UPDATE：

在对一条记录做UPDATE操作时分为三种情况：

- 1、如果未修改该记录的键值并且被更新的列占用的存储空间在**修改前后未发生变化**，则先在B+树中定位到这条记录的位置，然后再获取一下记录的X锁，最后在原记录的位置进行修改操作。其实我们也可以把这个定位待修改记录在B+树中位置的过程看成是一个**获取X锁的锁定读**。
- 2、如果未修改该记录的键值并且至少有一个被更新的列占用的存储空间在**修改前后发生变化**，则先在B+树中定位到这条记录的位置，然后获取一下记录的X锁，将该记录彻底删除掉（就是把记录彻底移入垃圾链表），最后再插入一条新记录。这个定位待修改记录在B+树中位置的过程看成是一个**获取X锁的锁定读**，新插入的记录由INSERT操作提供的**隐式锁进行保护**。
- 3、如果修改了该记录的键值，则相当于在原记录上做DELETE操作之后再进行一次INSERT操作，加锁操作就需要按照DELETE和INSERT的规则进行了。

1.6.4锁的粒度

我们前边提到的锁都是针对记录的，也可以被称之为行级锁或者行锁，对一条记录加锁影响的也只是这条记录而已，我们就说这个锁的粒度比较细；其实一个事务也可以在表级别进行加锁，自然就被称之为表级锁或者表锁，对一个表加锁影响整个表中的记录，我们就说这个锁的粒度比较粗。给表加的锁也可以分为共享锁（S锁）和独占锁（X锁）

1.6.4.1.表锁与行锁的比较

锁定粒度：表锁 > 行锁

加锁效率：表锁 > 行锁

冲突概率：表锁 > 行锁

并发性能：表锁 < 行锁

1.6.4.2.给表加S锁

如果一个事务给表加了S锁，那么：

别的事务可以继续获得该表的S锁

别的事务可以继续获得该表中的某些记录的S锁

别的事务不可以继续获得该表的X锁

别的事务不可以继续获得该表中的某些记录的X锁

1.6.4.3.给表加X锁

如果一个事务给表加了X锁（意味着该事务要独占这个表），那么：

别的事务不可以继续获得该表的S锁

别的事务不可以继续获得该表中的某些记录的S锁

别的事务不可以继续获得该表的X锁

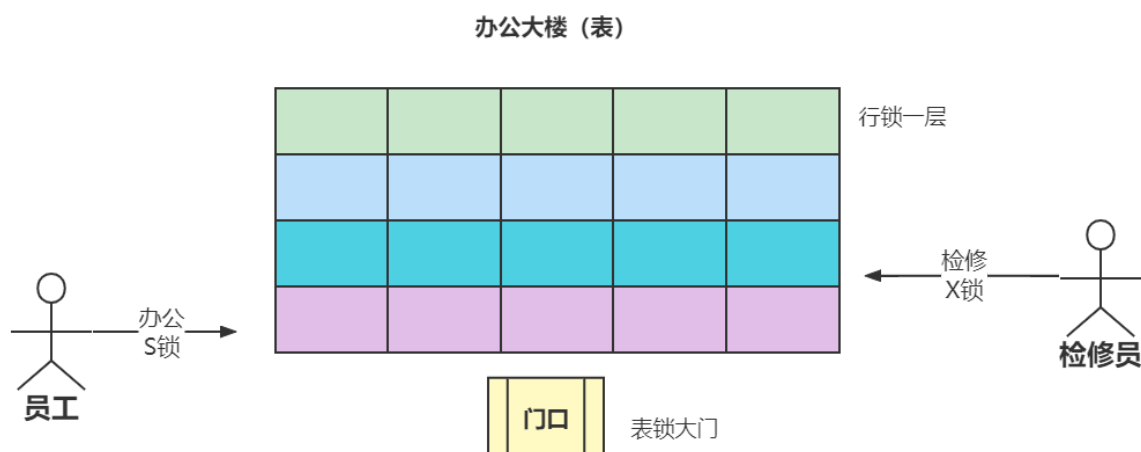
别的事务不可以继续获得该表中的某些记录的X锁。

为了更好的理解这个表级别的S锁和X锁和后面的意向锁，我们举一个现实生活中的例子。我们用曾经很火爆的互联网风口项目共享Office来说明加锁：

共享Office有栋大楼，楼自然有很多层。办公室都是共享的，客户可以随便选办公室办公。每层楼可以容纳客户同时办公，每当一个客户进去办公，就相当于在每层的入口处挂了一把S锁，如果很多客户进去办公，相当于每层的入口处挂了很多把S锁（类似行级别的S锁）。

有的时候楼层会进行检修，比方说换地板，换天花板，检查水电啥的，这些维修项目并不能同时开展。如果楼层针对某个项目进行检修，就不允许客户来办公，也不允许其他维修项目进行，此时相当于楼层门口会挂一把X锁（类似行级别的X锁）。

上边提到的这两种锁都是针对楼层而言的，不过有时候我们会有一些特殊的需求：



A、有投资人要来考察Office的环境。

投资人和公司并不想影响客户进去办公，但是此时不能有楼层进行检修，所以可以在大楼门口放置一把S锁（类似表级别的S锁）。此时：

来办公的客户们看到大楼门口有S锁，可以继续进入大楼办公。

修理工看到大楼门口有S锁，则先在大楼门口等着，啥时候投资人走了，把大楼的S锁撤掉再进入大楼维修。

B、公司要和房东谈条件。

此时不允许大楼中有正在办公的楼层，也不允许对楼层进行维修。所以可以在大楼门口放置一把X锁（类似表级别的X锁）。此时：

来办公的客户们看到大楼门口有X锁，则需要在大楼门口等着，啥时候条件谈好，把大楼的X锁撤掉再进入大楼办公。

修理工看到大楼门口有X锁，则先在大楼门口等着，啥时候谈判结束，把大楼的X锁撤掉再进入大楼维修。

1.6.5.意向锁 (InnoDB)

但是在上面的例子这里头有两个问题：

如果我们想对大楼整体上S锁，首先需要确保大楼中的没有正在维修的楼层，如果有正在维修的楼层，需要等到维修结束才可以对大楼整体上S锁。

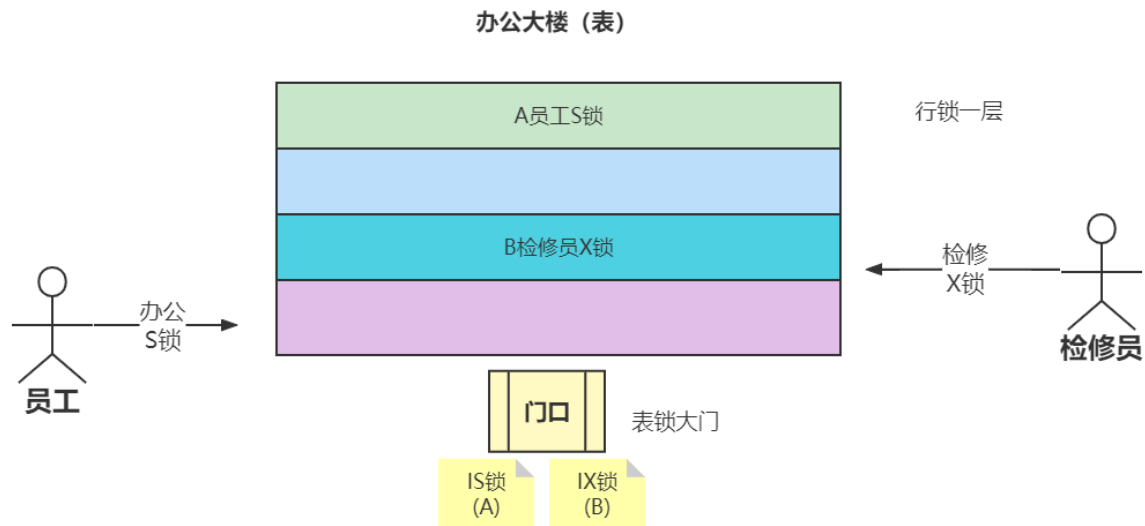
如果我们想对大楼整体上X锁，首先需要确保大楼中的没有办公的楼层以及正在维修的楼层，如果有办公的楼层或者正在维修的楼层，需要等到全部办公的同学都办公离开，以及维修工维修完楼层离开后才可以对大楼整体上X锁。

我们在对大楼整体上锁（表锁）时，怎么知道大楼中有没有楼层已经被上锁（行锁）了呢？依次检查每一楼层门口有没有上锁？那这效率也太慢了吧！于是InnoDB提出了一种意向锁（英文名：Intention Locks）：

意向共享锁，英文名：Intention Shared Lock，简称IS锁。当事务准备在某条记录上加S锁时，需要先在表级别加一个IS锁。

意向独占锁，英文名：Intention Exclusive Lock，简称IX锁。当事务准备在某条记录上加X锁时，需要先在表级别加一个IX锁。

视角回到大楼和楼层上来：



如果有客户到楼层中办公，那么他先在整栋大楼门口放一把IS锁（表级锁），然后再到楼层门口放一把S锁（行锁）。

如果有维修工到楼层中维修，那么它先在整栋大楼门口放一把IX锁（表级锁），然后再到楼层门口放一把X锁（行锁）。

之后：

如果有投资人要参观大楼，也就是想在大楼门口前放S锁（表锁）时，首先要看一下大楼门口有没有IX锁，如果有，意味着有楼层在维修，需要等到维修结束把IX锁撤掉后才可以在整栋大楼上加S锁。

如果有谈条件要占用大楼，也就是想在大楼门口前放X锁（表锁）时，首先要看一下大楼门口有没有IS锁或IX锁，如果有，意味着有楼层在办公或者维修，需要等到客户们办完公以及维修结束把IS锁和IX锁撤掉后才可以在整栋大楼上加X锁。

注意：客户在大楼门口加IS锁时，是不关心大楼门口是否有IX锁的，维修工在大楼门口加IX锁时，是不关心大楼门口是否有IS锁或者其他IX锁的。IS和IX锁只是为了判断当前时间大楼里有没有被占用的楼层用的，也就是在对大楼加S锁或者X锁时才会用到。

总结一下：IS、IX锁是表级锁，它们的提出仅仅为了在之后加表级别的S锁和X锁时可以快速判断表中的记录是否被上锁，以避免用遍历的方式来查看表中有没有上锁的记录。就是说其实IS锁和IX锁是兼容的，IX锁和IX锁是兼容的。我们画个表来看一下**表级别**的各种锁的兼容性：

兼容性	X	IX	S	IS
X	不兼容	不兼容	不兼容	不兼容
IX	不兼容		不兼容	
S	不兼容	不兼容		
IS	不兼容			

锁的组合性：（意向锁没有行锁）

组合性	X	IX	S	IS
表锁	有	有	有	有
行锁	有		有	

1.6.6.MySQL中的行锁和表锁

MySQL支持多种存储引擎，不同存储引擎对锁的支持也是不一样的。当然，我们重点还是讨论InnoDB存储引擎中的锁，其他的存储引擎只是稍微看看。

1.6.6.1.其他存储引擎中的锁

对于MyISAM、MEMORY、MERGE这些存储引擎来说，它们只支持表级锁，而且这些引擎并不支持事务，所以使用这些存储引擎的锁一般都是针对当前会话来说的。比方说在Session 1中对一个表执行SELECT操作，就相当于为这个表加了一个表级别的S锁，如果在SELECT操作未完成时，Session 2中对这个表执行UPDATE操作，相当于要获取表的X锁，此操作会被阻塞，直到Session 1中的SELECT操作完成，释放掉表级别的S锁后，Session 2中对这个表执行UPDATE操作才能继续获取X锁，然后执行具体的更新语句。

因为使用MyISAM、MEMORY、MERGE这些存储引擎的表在同一时刻只允许一个会话对表进行写操作，所以这些存储引擎实际上最好用在只读，或者大部分都是读操作，或者单用户的情景下。

另外，在MyISAM存储引擎中有一个称之为Concurrent Inserts的特性，支持在对MyISAM表读取时同时插入记录，这样可以提升一些插入速度。关于更多Concurrent Inserts的细节，详情可以参考文档。

1.6.6.2.InnoDB存储引擎中的锁

InnoDB存储引擎既支持表锁，也支持行锁。表锁实现简单，占用资源较少，不过粒度很粗，有时候你仅仅需要锁住几条记录，但使用表锁的话相当于为表中的所有记录都加锁，所以性能比较差。行锁粒度更细，可以实现更精准的并发控制。下边我们详细看一下。

1.6.6.2.1.InnoDB中的表级锁

1.6.6.2.1.1.表级别的S锁、X锁、元数据锁

在对某个表执行SELECT、INSERT、DELETE、UPDATE语句时，InnoDB存储引擎是不会为这个表添加表级别的S锁或者X锁的。

另外，在对某个表执行一些诸如ALTER TABLE、DROP TABLE这类的DDL语句时，其他事务对这个表并发执行诸如SELECT、INSERT、DELETE、UPDATE的语句会发生阻塞，同理，某个事务中对某个表执行SELECT、INSERT、DELETE、UPDATE语句时，在其他会话中对这个表执行DDL语句也会发生阻塞。这个过程其实是通过在server层使用一种称之为元数据锁（英文名：Metadata Locks，简称MDL）来实现的，一般情况下也不会使用InnoDB存储引擎自己提供的表级别的S锁和X锁。

其实这个InnoDB存储引擎提供的表级S锁或者X锁是相当鸡肋，只会是一些特殊情况下，比方说崩溃恢复过程中用到。不过我们还是可以手动获取一下的，比方说在系统变量autocommit=0，innodb_table_locks = 1时，手动获取InnoDB存储引擎提供的表t的S锁或者X锁可以这么写：

```
LOCK TABLES t
```

READ：InnoDB存储引擎会对表t加表级别的S锁。

```
LOCK TABLES t
```

WRITE：InnoDB存储引擎会对表t加表级别的X锁。

请尽量避免在使用InnoDB存储引擎的表上使用LOCK TABLES这样的手动锁表语句，它们并不会提供什么额外的保护，只是会降低并发能力而已。

1.6.6.2.1.2.表级别的IS锁、IX锁

当我们在对使用InnoDB存储引擎的表的某些记录加S锁之前，那就需要先在表级别加一个IS锁，当我们在对使用InnoDB存储引擎的表的某些记录加X锁之前，那就需要先在表级别加一个IX锁。

IS锁和IX锁的使命只是为了后续在加表级别的S锁和X锁时判断表中是否有已经被加锁的记录，以避免用遍历的方式来查看表中有没有上锁的记录。我们并不能手动添加意向锁，只能由InnoDB存储引擎自行添加。

1.6.6.2.1.3.表级别的AUTO-INC锁

在使用MySQL过程中，我们可以为表的某个列添加AUTO_INCREMENT属性，之后在插入记录时，可以不指定该列的值，系统会自动为它赋上递增的值系统实现这种自动给AUTO_INCREMENT修饰的列递增赋值的原理主要是两个：

1、采用AUTO-INC锁，也就是在执行插入语句时就在表级别加一个AUTO-INC锁，然后为每条待插入记录的AUTO_INCREMENT修饰的列分配递增的值，在该语句执行结束后，再把AUTO-INC锁释放掉。这样一个事务在持有AUTO-INC锁的过程中，其他事务的插入语句都要被阻塞，可以保证一个语句中分配的递增值是连续的。

如果我们的插入语句在执行前不可以确定具体要插入多少条记录（无法预计即将插入记录的数量），比方说使用INSERT ... SELECT、REPLACE ... SELECT或者LOAD DATA这种插入语句，一般是使用AUTO-INC锁为AUTO_INCREMENT修饰的列生成对应的值。

2、采用一个轻量级的锁，在为插入语句生成AUTO_INCREMENT修饰的列的值时获取一下这个轻量级锁，然后生成本次插入语句需要用到的AUTO_INCREMENT列的值之后，就把该轻量级锁释放掉，并不需要等到整个插入语句执行完才释放锁。

如果我们的插入语句在执行前就可以确定具体要插入多少条记录，比方说我们上边举的关于表t的例子中，在语句执行前就可以确定要插入2条记录，那么一般采用轻量级锁的方式对AUTO_INCREMENT修饰的列进行赋值。这种方式可以避免锁定表，可以提升插入性能。

InnoDB提供了一个称之为innodb_autoinc_lock_mode的系统变量来控制到底使用上述两种方式中的哪种来为AUTO_INCREMENT修饰的列进行赋值，当innodb_autoinc_lock_mode值为0时，一律采用AUTO-INC锁；当innodb_autoinc_lock_mode值为1时，一律采用轻量级锁；当innodb_autoinc_lock_mode值为2时，两种方式混着来（也就是在插入记录数量确定时采用轻量级锁，不确定时使用AUTO-INC锁）。

不过当innodb_autoinc_lock_mode值为2时，可能会造成不同事务中的插入语句为AUTO_INCREMENT修饰的列生成的值是交叉的，在有主从复制的场景中是不安全的。

```
show variables like 'innodb_autoinc_lock_mode' ;
```

```
mysql> show variables like 'innodb_autoinc_lock_mode' ;
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_autoinc_lock_mode | 1     |
+-----+-----+
1 row in set (0.00 sec)
```

MySQL5.7.X中缺省为1。

1.6.7.2.InnoDB中的行级锁

行锁，也称为记录锁，顾名思义就是在记录上加的锁。但是要注意，这个记录指的是通过给索引上的索引项加锁。InnoDB 这种行锁实现特点意味着：**只有通过索引条件检索数据，InnoDB 才使用行级锁，否则，InnoDB 将使用表锁。**

不论是使用主键索引、唯一索引或普通索引，InnoDB都会使用行锁来对数据加锁。

只有执行计划真正使用了索引，才能使用行锁：**即便在条件中使用了索引字段，但是否使用索引来检索数据是由 MySQL 通过判断不同执行计划的代价来决定的，如果 MySQL认为全表扫描效率更高，比如对一些很小的表，它就不会使用索引，这种情况下 InnoDB 将使用表锁，而不是行锁。**

同时当我们用范围条件而不是相等条件检索数据，并请求锁时，InnoDB会给符合条件的已有数据记录的索引项加锁。

不过即使是行锁，InnoDB里也是分成了各种类型的。换句话说即使对同一条记录加行锁，如果类型不同，起到的功效也是不同的。我们使用前面的teacher，增加一个索引，并插入几条记录。

```
INDEX `idx_number`(`number`)
```

```
mysql> select * from teacher;
+-----+-----+-----+
| number | name   | domain      |
+-----+-----+-----+
| 1      | 李瑾   | JVM系列     |
| 2      | 严镇涛 | SpringCloud |
| 6      | 马老师 | 并发编程    |
| 10     | 连老师 | 并发编程    |
| 30     | 豹     | 数据湖      |
+-----+-----+-----+
```

我们来看看都有哪些常用的行锁类型。

Record Locks

也叫记录锁，就是仅仅把一条记录锁上，官方的类型名称为：LOCK_REC_NOT_GAP。比方说我们把number值为6的那条记录加一个记录锁的示意图如下：

number	name	domain
1	李瑾	JVM系列
2	严镇涛	SpringCloud
6	马老师	并发编程
10	连老师	Spring源码
30	豹	数据湖

记录锁是有S锁和X锁之分的，当一个事务获取了一条记录的S型记录锁后，其他事务也可以继续获取该记录的S型记录锁，但不可以继续获取X型记录锁；当一个事务获取了一条记录的X型记录锁后，其他事务既不可以继续获取该记录的S型记录锁，也不可以继续获取X型记录锁；

Gap Locks 间隙锁

我们说MySQL在REPEATABLE READ隔离级别下是可以解决幻读问题的，解决方案有两种，可以使用MVCC方案解决，也可以采用加锁方案解决。但是在使用加锁方案解决时有问题，就是事务在第一次执行读取操作时，那些幻影记录尚不存在，我们无法给这些幻影记录加上记录锁。InnoDB提出了一种称之为Gap Locks的锁，官方的类型名称为：LOCK_GAP，我们也可以简称为gap锁。

间隙锁实质上是对索引前后的间隙上锁，不对索引本身上锁。

会话1开启一个事务，执行

```
begin;  
update teacher set domain = 'JVM' where number = '6';
```

会对2~6之间和6到10之间进行上锁。

number	name	domain
1	李瑾	JVM系列
2	严镇涛	SpringCloud
6	马老师	并发编程
10	连老师	Spring源码
30	豹	数据湖

如图中为2~6和 6 ~ 10的记录加了gap锁，意味着不允许别的事务在这条记录前后间隙插入新记录。

```
begin;
insert into teacher value(7,'晁','docker');
```

为什么不能插入？因为记录（7,'晁','docker'）要插入的话，在索引idx_number上，刚好落在6 ~ 10之间，是有锁的，当然不允许插入。

但是当SQL语句变为：insert
into teacher value(70,'晁','docker');能插入吗？

当然能，因为70这条记录不在被锁的区间内。

1.6.7.死锁

1.6.7.1.概念

是指两个或两个以上的进程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，若无外力作用，它们都将无法推进下去。此时称系统处于死锁状态或系统产生了死锁。

举个例子：A和B去按摩洗脚，都想在洗脚的时候，同时顺便做个头部按摩，13技师擅长足底按摩，14擅长头部按摩。

这个时候A先抢到14，B先抢到13，两个人都想同时洗脚和头部按摩，于是就互不相让，扬言我死也不让你，这样的话，A抢到14，想要13，B抢到13，想要14，在这个想同时洗脚和头部按摩的事情上A和B就产生了死锁。怎么解决这个问题呢？

第一种，假如这个时候，来了个15，刚好也是擅长头部按摩的，A又没有两个脑袋，自然就归了B，于是B就美滋滋的洗脚和做头部按摩，剩下A在旁边气鼓鼓的，这个时候死锁这种情况就被打破了，不存在了。

第二种，C出场了，用武力强迫A和B，必须先做洗脚，再头部按摩，这种情况下，A和B谁先抢到13，谁就可以进行下去，另外一个没抢到的，就等着，这种情况下，也不会产生死锁。

所以总结一下：

死锁是必然发生在多操作者（ $M \geq 2$ 个）情况下，争夺多个资源（ $N \geq 2$ 个，且 $N \leq M$ ）才会发生这种情况。很明显，单线程自然不会有死锁，只有B一个去，不要2个，打十个都没问题；单资源呢？只有13，A和B也只会产生激烈竞争，打得不可开交，谁抢到就是谁的，但不会产生死锁。同时，死锁还有几个要求，1、争夺资源的顺序不对，如果争夺资源的顺序是一样的，也不会产生死锁；

2、争夺者拿到资源不放手。

1.6.7.2.MySQL中的死锁

MySQL中的死锁的成因是一样的。

会话1：

```
begin;
select * from
teacher where number = 1 for update;
```

会话2：

```
begin;
select * from
teacher where number = 3 for update;
```

会话1

```
select * from teacher where number = 3 for
update;
```

可以看到这个语句的执行将会被阻塞

会话2：

```
select * from
teacher where number = 1 for update;
```

MySQL检测到了死锁，并结束了会话2中事务的执行，此时，切回会话1，发现原本阻塞的SQL语句执行完成了。

同时通过

```
show engine innodb status\G
```

可以看见死锁的详细情况：

```
LATEST DETECTED DEADLOCK
-----
2021-04-14 12:19:37 0x7fadfde65700
*** (1) TRANSACTION:
TRANSACTION 14037, ACTIVE 41 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1136, 2 row lock(s)
MySQL thread id 124, OS thread handle 140385570518784, query id 5876 localhost root statisc
select * from teacher where number = 3 for update
*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 33 page no 3 n bits 80 index PRIMARY of table `mysqladv`.`teacher` t
Record lock, heap no 3 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
 0: len 4; hex 80000003; asc      ;;
 1: len 6; hex 00000000321d; asc    2 ;;
 2: len 7; hex ba0000012f0110; asc  /  ;;
 3: len 4; hex 4d61726b; asc Mark;;
 4: len 12; hex e5b9b6e58f91e7bc96e7a88b; asc      ;;

*** (2) TRANSACTION:
TRANSACTION 14038, ACTIVE 29 sec starting index read
mysql tables in use 1, locked 1
3 lock struct(s), heap size 1136, 2 row lock(s)
MySQL thread id 123, OS thread handle 140385265800960, query id 5877 localhost root statisc
select * from teacher where number = 1 for update
*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 33 page no 3 n bits 80 index PRIMARY of table `mysqladv`.`teacher` t
Record lock, heap no 3 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
 0: len 4; hex 80000003; asc      ;;
 1: len 6; hex 00000000321d; asc    2 ;;
 2: len 7; hex ba0000012f0110; asc  /  ;;
 3: len 4; hex 4d61726b; asc Mark;;
 4: len 12; hex e5b9b6e58f91e7bc96e7a88b; asc      ;;

*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 33 page no 3 n bits 80 index PRIMARY of table `mysqladv`.`teacher` t
Record lock, heap no 2 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
 0: len 4; hex 80000001; asc      ;;
 1: len 6; hex 00000000321c; asc    2 ;;
 2: len 7; hex b90000012d0110; asc  -  ;;
 3: len 4; hex 4a61636b; asc Jack;;
 4: len 12; hex e6ba90e7a081e7b3bbe58897; asc      ;;

*** WE ROLL BACK TRANSACTION (2)
```

查看事务加锁的情况，不过一般情况下，看不到哪个事务对哪些记录加了那些锁，需要修改系统变量 innodb_status_output_locks (MySQL5.6.16引入)，缺省是OFF。

```
show
variables like 'innodb_status_output_locks';
```

```
mysql> show variables like 'innodb_status_output_locks';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_status_output_locks | OFF   |
+-----+-----+
1 row in set (0.00 sec)
```

我们需要设置为ON，

```
set global
innodb_status_output_locks = ON;
```

然后开启事务，并执行语句